

從氛圍檢查到以資料為依據的服務專員評估

來源：<https://codelabs.developers.google.com/codelabs/production-ready-ai-roadshow/2-evaluating-multi-agent-systems/evaluating-multi-agent-systems?hl=zh-tw>

步驟數：10

「從 Vibe Checks 到資料驅動的虛擬服務專員評估：透過 Cloud Run 上的 Vertex AI 實現」

目錄

1. [1. 簡介](#)
2. [2. 核心概念：代理評估理論](#)
3. [3. 設定](#)
4. [4. 瞭解安全部署](#)
5. [5. 導入評估指令碼](#)
6. [6. 瞭解評估程式碼](#)
7. [7. 執行評估作業](#)
8. [8. 在筆記本中以圖表呈現結果](#)
9. [9. 持續整合與部署 \(CI/CD\)](#)
10. [10. 摘要](#)

1. 簡介

總覽

本實驗室是「[使用 ADK 建構多代理系統](#)」的後續課程。

在該實驗室中，您建構了課程建立系統，其中包含：

1. **研究人員代理**：使用 `google_search` 尋找最新資訊。
2. **Judge 代理**：評估研究的品質和完整性。
3. **內容建立工具代理程式**：將研究結果轉換為結構化課程。
4. **自動化調度管理工具代理**：管理工作流程，以及這些專家之間的通訊。

這項服務也包含網頁應用程式，可讓使用者提交課程建立要求，並取得課程做為回應。

研究人員、評估人員和內容建構工具會以 [A2A 代理程式](#) 的形式，部署在不同的 [Cloud Run](#) 服務中。**Orchestrator** 是另一個具有 ADK Service API 的 [Cloud Run](#) 服務。

在本實驗室中，我們修改了 **Researcher** 代理程式，讓它使用 **Wikipedia Search** 工具，而非 Gemini 的 **Google 搜尋** 功能。方便我們檢查自訂工具呼叫的追蹤和評估方式。

因此，我們建構了分散式多代理系統。但要如何評估實際成效呢？研究人員是否一律會找到相關資訊？評估人員是否正確識別不良研究？

在本實驗室中，您將使用 [Vertex AI Gen AI Evaluation Service](#)，以資料導向的評估方式取代主觀的「感覺檢查」。您將導入自動調整評分量表和工具使用品質指標，嚴格評估在實驗室 1 中建構的分散式多代理系統。最後，您會在 CI/CD 管道中自動執行這項程序，確保每次部署作業都能維持正式環境代理程式的可靠性和準確度。

您將為代理建構**持續評估管道**。您將學習下列內容：

1. 將代理程式部署至 Google Cloud Run 中的私有**標記修訂版本** (影子部署)。
 2. 使用 [Vertex AI Gen AI Evaluation Service](#)，針對特定修訂版本執行自動評估套件。
 3. 以圖表呈現結果並加以分析。
 4. 將評估結果做為 CI/CD 管道的一部分。
-

2. 核心概念：代理評估理論

開發及執行 AI 代理程式時，我們會進行兩種評估：**離線實驗**和**透過自動迴歸測試持續評估**。首先是開發過程的創意引擎，我們會執行臨時實驗、修正提示，並快速疊代，以解鎖新功能。第二層是 CI/CD 管道中的防禦層，我們會針對「黃金」資料集執行持續評估，確保程式碼變更不會無意間降低代理程式的品質。

兩者的根本差異在於「探索」與「防禦」：

- **離線實驗**是最佳化程序，這項功能沒有固定時間，您正在積極變更輸入內容 (提示詞、模型、參數)，以盡可能提高分數或解決特定問題。目標是提高代理程式可執行的「上限」。
- **持續評估 (自動迴歸測試)** 是驗證程序，這種做法既僵化又重複。您會將輸入內容保持不變 (「黃金」資料集)，確保輸出內容維持穩定。目標是防止成效「下限」崩盤。

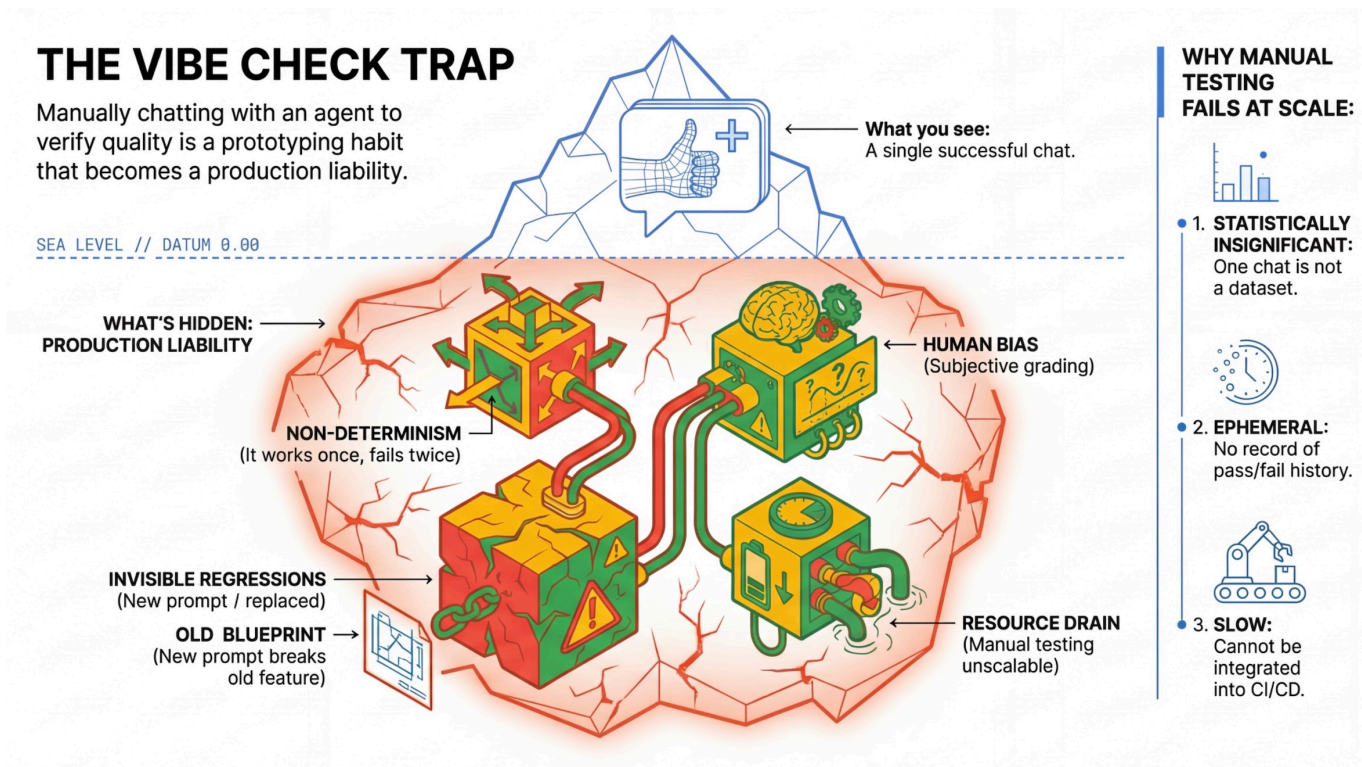
在本實驗室中，我們將著重於**持續評估**。我們會開發自動迴歸測試管道，在有人變更 AI 代理程式時執行，就像單元測試一樣。

編寫程式碼前，請務必瞭解要評估的內容。

「氛圍檢查」陷阱

許多開發人員會手動與代理程式對話，藉此測試代理程式。這就是所謂的「氛圍檢查」。雖然這項功能有助於製作原型，但無法用於正式版，原因如下：

- **非確定性**：代理每次的回覆可能不同。您需要具備統計顯著性的樣本量。
- **隱形回歸**：改善某個提示可能會導致其他用途無法正常運作。
- **人類偏誤**：「看起來不錯」是主觀判斷。
- **耗時的工作**：每次提交時，手動測試數十種情境會很慢。



評估代理程式效能的兩種方式

為建構強大的管道，我們結合了不同類型的評分者：

1. 以程式碼為準的評分者 (確定性)：

- **評估內容**：嚴格限制 (例如「是否傳回有效的 JSON？」、「Did it call the `search` tool?» (是否呼叫 `search` 工具?))。
- **優點**：速度快、成本低，準確度達 100%。
- **缺點**：無法判斷細微差異或品質。

2. 以模型為基礎的評分者 (機率)：

- 又稱「LLM-as-a-Judge」。我們會使用強大模型 (例如 Gemini 3 Pro) 評估代理程式的輸出內容。
- **評估內容**：細微差異、推論、實用性、安全性。
- **優點**：可評估複雜的開放式工作。
- **缺點**：速度較慢、成本較高，需要仔細設計評估人員的提示。

Vertex AI 評估指標

在本實驗室中，我們使用 **Vertex AI Gen AI Evaluation Service**，這項服務提供受管理的指標，因此您不必從頭編寫每個評估人員。

您可以透過多種方式分組代理評估指標：

- **以評分量表為基礎的指標**：將 LLM 納入評估工作流程。
 - **自動調整評量表**：系統會為每個提示動態生成評量表。系統會根據提示詞，以精細且可解釋的通過或失敗回饋評估回覆。
 - **靜態評量表**：明確定義評量表，並將同一份評量表套用至所有提示詞。系統會使用同一組以分數為依據的評估人員來評估回覆。每個提示詞的單一數字分數 (例如 1 到 5 分)。需要評估非常具體的維度，或所有提示都必須使用完全相同的評分標準時。
- **以運算為基礎的指標**：使用確定性演算法評估回覆，通常會使用基準真相。每個提示詞的數值分數 (例如 0.0 到 1.0)。實際資料可用，且可透過確定性方法比對。
- **自訂函式指標**：透過 Python 函式定義自己的指標。

我們將使用的具體指標：

- **Final Response Match**：(以參照內容為準) 答案是否符合我們的「黃金答案」？
 - **Tool Use Quality**：(不參考) 服務專員是否以適當方式使用相關工具？
 - **Hallucination**：(不含參照) 擷取的背景資訊是否支援回覆中的聲明？
 - **Tool Trajectory Precision** 和 **Tool Trajectory Recall** (以參照為準)：代理是否選取正確的工具並提供有效引數？與 **Tool Use Quality** 不同，這些自訂指標會使用參考軌跡，也就是預期工具呼叫和引數的序列。
-

步驟 3 · 約 0 分鐘

3. 設定

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

設定

1. 開啟 **Cloud Shell**：點選 Google Cloud 控制台右上方的「啟用 Cloud Shell」圖示。
2. 執行下列指令，重新整理登入狀態並更新應用程式預設憑證 (ADC)：

```
gcloud auth login --update-adc
```

按照操作說明在瀏覽器中完成登入。

3. 為 gcloud CLI 設定有效專案。執行下列指令，取得目前的 gcloud 專案：

```
gcloud config get-value project
```

如果未設定，請執行下列指令：

```
gcloud config set project YOUR_PROJECT_ID
```

將 `YOUR_PROJECT_ID` 替換為專案 ID。

4. 設定要部署 Cloud Run 服務的預設區域。

```
gcloud config set run/region us-west1
```

您可以改用離您較近的任何 [Cloud Run 區域](#)。 `us-west1`

程式碼和依附元件

1. 複製範例程式碼，然後切換到專案根目錄。

```
cd ~
git clone https://github.com/vladkol/agent-evaluation-lab -b starter
cd agent-evaluation-lab
```

2. 建立 `.env` 檔案：

```
echo "GOOGLE_GENAI_USE_VERTEXAI=true" > .env
echo "GOOGLE_CLOUD_PROJECT=$(gcloud config get-value project -q)" >> .env
echo "GOOGLE_CLOUD_REGION=$(gcloud config get-value run/region -q)" >> .env
echo "GOOGLE_CLOUD_LOCATION=global" >> .env
```

3. 在終端機視窗中執行下列指令，安裝依附元件：

```
uv sync
```

步驟 4 · 約 0 分鐘

4. 瞭解安全部署

我們需要先部署，才能進行評估。但如果新程式碼有問題，我們不希望導致上線中的應用程式中斷。

修訂版本標記和陰影部署

Google Cloud Run 支援修訂版本。每次部署時，系統都會建立新的不可變更修訂版本。您可以為這些修訂版本指派標記，透過特定網址存取這些版本，即使這些版本接收的公開流量為 0% 也是如此。

為何不直接在本地執行評估？

雖然 ADK 支援本機評估，但部署至隱藏修訂版本可為生產系統帶來重大優勢。這可區分系統層級評估 (我們目前執行的作業) 和單元測試：

1. **環境同等性**：本機環境不同 (網路、CPU/記憶體、密鑰都不同)。在雲端中測試可確保代理程式在實際執行階段環境 (系統測試) 中運作。
2. **多代理互動**：在分散式系統中，代理程式會透過 HTTP 通訊。「本機」測試通常會模擬這些連線。影子部署會測試微服務之間的實際網路延遲時間、逾時設定和驗證。
3. **密鑰和權限**：確認服務帳戶是否具備所需權限 (例如呼叫 Vertex AI 或從 Firestore 讀取資料)。

附註：這是**主動評估** (在使用者看到內容前進行檢查)。部署完成後，您可以使用**被動監控** (可觀測性) 找出實際環境中的問題。

CI/CD 工作流程：部署、評估、升級

我們使用這項功能來建立穩固的**持續部署**管道：

1. **提交**：變更代理程式的提示，並推送到存放區。
2. **部署 (隱藏)**：觸發部署標有提交雜湊的新修訂版本 (例如 `c-abc1234`)。這個修訂版本會收到 **0%** 的公開流量。
3. **評估**：評估指令碼會以特定修訂網址 `https://c-abc1234---researcher-xyz.run.app` 為目標。
4. **升級**：如果 (且僅在) 評估通過且其他測試成功，請**遷移流量**至這個新修訂版本。
5. **復原**：如果失敗，使用者從未看過錯誤版本，您可以直接忽略或刪除錯誤修訂版本。

這項策略可讓您在實際工作環境中進行測試，而不影響客戶。

分析 `evaluate.sh`

開啟 `evaluate.sh`。這個指令碼會自動執行程序。

```
export COMMIT_SHORT_HASH=$(git rev-parse --short HEAD)
export COMMIT_REVISION_TAG="c-${COMMIT_SHORT_HASH}"

# ...

# Deploy services with a revision tag and NO traffic
source ./deploy.sh --revision-tag $COMMIT_REVISION_TAG --no-redeploy

# Run the evaluation against that specific tag
uv run -m evaluator.evaluate_agent
```

`deploy.sh` 會透過 `--no-traffic` 和 `--tag` 選項處理修訂版本部署作業。如果已有服務正在執行，則不會受到影響。除非您使用含有修訂版本標記 (例如 `https://c-abc1234---researcher-xyz.run.app`) 的特殊網址明確呼叫，否則新的「隱藏」修訂版本不會收到任何流量

5. 導入評估指令碼

現在，我們來編寫實際執行測試的程式碼。

1. 開啟 `evaluator/evaluate_agent.py`。
2. 您會看到匯入和設定，但缺少指標和執行邏輯。

定義指標

對於研究人員代理程式，我們有「黃金答案」/「基準真相」和預期答案。這是「能力評估」：我們正在評估代理程式是否正確完成工作。

我們想評估：

- **最終回覆相符**：(能力) 答案是否符合預期？這是以參照為基礎的指標。系統會使用評估 LLM，比較代理的輸出內容與預期答案。這項指標並非要求答案完全相同，而是語意和事實相似。
- **工具使用品質**：(品質) 這項適用於特定目標的自動調整評分量表指標會評估所選工具是否適當、參數使用方式是否正確，以及是否符合指定的作業順序。
- **工具使用軌跡**：(追蹤) 2 個自訂指標，用於根據預期軌跡，評估代理程式的工具使用軌跡 (精確度和召回率)。這些指標會在 `shared/evaluation/tool_metrics.py` 中以自訂函式實作。與**工具使用品質**不同，這項指標是以參考資料為依據的確定性指標，程式碼會實際查看工具呼叫是否與參考資料 (評估資料中的 `reference_trajectory`) 相符。

自訂工具使用軌跡指標

我們在 `shared/evaluation/tool_metrics.py` 中建立了一組 Python 函式，用於自訂「工具使用軌跡」指標。如要允許 Vertex AI Gen AI Evaluation Service 執行這些函式，我們需要將該 Python 程式碼傳遞給該服務。

方法是定義具有 `UnifiedMetric` 和 `CustomCodeExecutionSpec` 設定的 `EvaluationRunMetric` 物件。參數 `remote_custom_function` 是包含函式 Python 程式碼的字串。函式必須命名為 `evaluate`：

```
def evaluate(  
    instance: dict  
) -> float:  
    ...
```

我們在 `shared/evaluation/evaluate.py` 中建立 `get_custom_function_metric` 輔助程式，可將 Python 函式轉換為自訂程式碼評估指標。

這個函式會取得函式模組的程式碼 (擷取本機依附元件)、建立呼叫原始函式的額外 `evaluate` 函式，並傳回含有 `CustomCodeExecutionSpec` 的 `EvaluationRunMetric` 物件。

```

import inspect
module_source = inspect.getsource(
    inspect.getmodule(metrics_function)
)
module_source += (
    "\n\ndef evaluate(instance: dict) -> float:\n"
    f"    return {metrics_function.__name__}(instance)\n"
)
return types.EvaluationRunMetric(
    metric=metric_name,
    metric_config=types.UnifiedMetric(
        custom_code_execution_spec=types.CustomCodeExecutionSpec(
            remote_custom_function=module_source
        )
    )
)
)

```

Gen AI Evaluation Service 會在沙箱執行環境中執行該程式碼，並將評估資料傳遞給該程式碼。

新增指標和評估程式碼

在 `if __name__ == "__main__":` 行之後，將下列程式碼新增至 `evaluator/evaluate_agent.py`。

定義 Researcher 代理的指標清單，並執行評估。

```

eval_data_researcher = os.path.dirname(__file__) + "/eval_data_researcher.json"
metrics=[
    # Compares the agent's output against a "Golden Answer"
    types.RubricMetric.FINAL_RESPONSE_MATCH,
    # Did the agent use the tools effectively?
    types.RubricMetric.TOOL_USE_QUALITY,
    # Custom metrics for tools trajectory analysis
    get_custom_function_metric("trajectory_precision", trajectory_precision_func),
    get_custom_function_metric("trajectory_recall", trajectory_recall_func)
]

print("🟢 Running Researcher Evaluation...")
eval_results = asyncio.run(
    # Run the evaluation and retrieve the results.
    evaluate_agent(
        agent_api_server=RESEARCHER_URL, # Agent Service URL (in Cloud Run).
        agent_name="agent", # Agent name as it's exposed by the server.
        evaluation_data_file=eval_data_researcher, # Evaluation data file.
        # GCS location for the Evaluation Service to store the result to.
        evaluation_storage_uri=f"gs://{GOOGLE_CLOUD_PROJECT}-agents/evaluation",
        metrics=metrics, # Metrics to use when evaluating the agent.
        project_id=GOOGLE_CLOUD_PROJECT,
        location=GOOGLE_CLOUD_REGION
    )
)
print(f"\n🟢 Researcher Evaluation results:\n{eval_results}")
print(f"Evaluation Run ID: {eval_results.run_id}")

```

在實際的正式管道中，您需要評估成功條件。評估完成且指標準備就緒後，您會在這裡看到「Gating Step」。例如：「If Final Response Match score < 0.75, fail the build.」（如果 Final Response Match 分數低於 0.75，則建構失敗）。這樣一來，不良修訂版本就不會收到流量。

將下列程式碼附加至 `evaluator/evaluate_agent.py`：

```

METRIC_THRESHOLD = 0.75
researcher_eval_failed = False
if eval_results.state != types.EvaluationRunState.SUCCEEDED:
    print(f"🔴 Researcher Evaluation failed with state {eval_results.state}.")
    researcher_eval_failed = True
else:
    for metric_name, metric_values in eval_results.metrics.items():
        if metric_values["mean"] < METRIC_THRESHOLD:
            print(f"🔴 Researcher Evaluation failed with metric `{metric_name}` below {METRIC_THRESHOLD} threshold.")
            researcher_eval_failed = True
if researcher_eval_failed:
    exit(1)

```

只要任何評估指標的**平均值**低於**門檻** (0.75)，就應部署失敗。

[選用] 使用無參考指標為自動調度管理工具新增評估

自動調度管理工具代理程式的互動較為複雜，有時可能沒有單一「正確」答案。我們會改用其中一個無參考指標評估一般行為。

- **幻覺**：這項分數型指標會檢查文字回覆內容是否符合事實且一致，做法是將回覆內容拆分為多個不可分割的部分，然後依據中繼事件中的工具使用情況，驗證各部分是否有憑有據。對於開放式代理程式而言，這點至關重要，因為「正確性」是主觀的，但「真實性」是無可妥協的。這項分數的計算方式是：以來源內容為依據的聲明所占百分比。在本例中，我們預期 Orchestrator (Content Builder 產生) 的最終回覆，會以 Researcher 使用維基百科搜尋工具擷取的內容為事實基礎。

新增 Orchestrator 的評估邏輯：

```
eval_data_orchestrator = os.path.dirname(__file__) + "/eval_data_orchestrator.json"
metrics=[
    types.RubricMetric.HALLUCINATION,
]

print("🟢 Running Orchestrator Evaluation...")
eval_results = asyncio.run(evaluate_agent(
    agent_api_server=ORCHESTRATOR_URL,
    agent_name="agent",
    evaluation_data_file=eval_data_orchestrator,
    evaluation_storage_uri=f"gs://{GOOGLE_CLOUD_PROJECT}-agents/evaluation",
    metrics=metrics,
    project_id=GOOGLE_CLOUD_PROJECT,
    location=GOOGLE_CLOUD_REGION
))
print(f"\n🟢 Orchestrator Evaluation results:\n{eval_results}")
print(f"Evaluation Run ID: {eval_results.run_id}")
METRIC_THRESHOLD = 0.75
orchestrator_eval_failed = False
if eval_results.state != types.EvaluationRunState.SUCCEEDED:
    print(f"🔴 Orchestrator Evaluation failed with state {eval_results.state}.")
    orchestrator_eval_failed = True
else:
    for metric_name, metric_values in eval_results.metrics.items():
        if metric_values["mean"] < METRIC_THRESHOLD:
            print(f"🔴 Orchestrator Evaluation failed with metric `{metric_name}` below {METRIC_THRESHOLD} threshold.")
            orchestrator_eval_failed = True
if orchestrator_eval_failed:
    exit(1)
```

檢查評估資料

開啟 `evaluator/` 目錄。您會看到兩個資料檔案：

- `eval_data_researcher.json`：研究人員的提示詞和黃金/基準真相參考資料。
- `eval_data_orchestrator.json`：自動化調度管理工具的提示 (我們只會對自動化調度管理工具執行無參考評估)。

每個項目通常包含：

- `prompt` : 代理程式的提示。
 - `reference` : 理想答案 (基準真相), 如適用。
 - `reference_trajectory` : 預期的工具呼叫順序。
-

6. 瞭解評估程式碼

開啟 `shared/evaluation/evaluate.py`。這個模組包含執行評估的核心邏輯。主要功能是 `evaluate_agent`。

這個檔案會執行下列步驟：

1. **載入資料**：從檔案讀取評估資料集 (提示和參考資料)。
2. **平行推論**：對資料集平行執行代理程式。這個類別會處理工作階段建立作業、傳送提示，並擷取最終回覆和**中介工具執行追蹤記錄**。
3. **Vertex AI Evaluation**：這項服務會合併原始評估資料、最終回覆和中介工具執行追蹤記錄，並透過 Vertex AI SDK 中的 GenAI Client，將結果提交至 [Vertex AI Evaluation Service](#)。這項服務會執行設定的指標，評估代理程式的成效。

最後一個步驟的關鍵時刻是呼叫 Gen AI SDK 評估模組的 `create_evaluation_run` 函式：

```
evaluation_run = client.evals.create_evaluation_run(  
    dataset=agent_dataset_with_inference,  
    agent_info=agent_info,  
    metrics=metrics,  
    dest=evaluation_storage_uri  
)
```

我們會在 `shared/evaluation/evaluate.py` 的 `evaluate_agent` 函式中執行這項操作。

這項作業會取得合併的評估資料集、代理程式相關資訊、要使用的指標，以及目的地儲存空間 URI。這個函式會在 Vertex AI Evaluation Service 中建立評估執行作業，並傳回評估執行作業物件。

Agent Info API

為進行準確的評估，評估服務需要瞭解代理程式的設定 (系統指令、說明和可用工具)。我們將其做為 `agent_info` 參數傳遞至 `create_evaluation_run`。

但我們如何取得這項資訊？我們將其納入 ADK 服務 API。

開啟 `shared/adk_app.py` 並搜尋 `def agent_info`。您會看到 ADK 應用程式公開輔助端點：

```
@app.get("/apps/{agent_name}/agent-info")  
async def agent_info(agent_name: str) -> typing.Dict[str, typing.Any]:  
    # ...  
    return {  
        "name": agent.name,  
        "instruction": str(getattr(agent, "instruction", None)),  
        "tool_declarations": tools_dict_list  
    }
```

這個端點 (透過 `--publish_agent_info` 標記啟用) 可讓評估指令碼動態擷取代理程式的執行階段設定。這對評估工具使用情況的指標至關重要，因為如果法官模型具體知道代理程式在對話期間可使用的工具，就能更妥善地評估代理程式的工具使用情況。

7. 執行評估作業

評估工具實作完成後，接著來執行！

1. 從存放區的根目錄執行評估指令碼：

```
./evaluate.sh
```

後續步驟

1. 取得目前的 Git 修訂版本雜湊碼。
2. 這會叫用 `deploy.sh`，根據修訂版本的雜湊值部署修訂版本。
3. 部署完成後，就會開始 `evaluator.evaluate_agent`。
4. 測試案例在雲端服務上執行時，您會看到進度列。
5. 最後，系統會列印結果的摘要 JSON。

執行指令碼時，可能會看到下列提示：

```
Deploying from source requires an Artifact Registry Docker repository to store  
built containers. A repository named [cloud-run-source-deploy] in region [us-west1]  
will be created.
```

```
Do you want to continue (Y/n)?
```

按下 <Enter> 鍵，允許建立存放區。

注意：首次執行時，部署服務可能需要幾分鐘。

8. 在筆記本中以圖表呈現結果

原始 JSON 輸出內容難以閱讀。Vertex AI SDK 中的 Gen AI 用戶端提供追蹤這些執行作業的方式。我們會使用 Colab 筆記本將結果視覺化。

1. 使用 `evaluator/show_evaluation_run.ipynb` [這個連結](#) 在 Google Colab 中開啟。
2. 將 `GOOGLE_CLOUD_PROJECT`、`GOOGLE_CLOUD_REGION` 和 `EVAL_RUN_ID` 變數設為專案 ID、區域和執行 ID。

`EVAL_RUN_ID` 是從執行 `./evaluate.sh` 的終端機記錄中取得的 **Evaluation Run ID** 值。

如果評估作業執行時發生本機執行階段回溯，請使用[這個 Google Colab](#)。

3. 安裝依附元件並進行驗證。

擷取評估執行作業並顯示結果

如果評估執行作業發生本機執行作業回溯，您只需要評估結果的 `gs://` URI。

我們需要從 Vertex AI 擷取評估執行資料。找出「Retrieve Evaluation Run and Display Results」(擷取評估執行作業並顯示結果) 下方的儲存格，然後將 `# TODO` 行替換為下列程式碼區塊：

```
from google.genai import types as genai_types
from vertexai import Client

# Initialize SDK
client = Client(
    project=GOOGLE_CLOUD_PROJECT,
    location=GOOGLE_CLOUD_REGION,
    http_options=genai_types.HttpOptions(api_version="v1beta1"),
)

evaluation_run = client.evals.get_evaluation_run(
    name=EVAL_RUN_ID,
    include_evaluation_items=True
)
evaluation_run.show()
```

解譯結果

查看結果時，請注意以下幾點：

1. 迴歸與功能：
 - **迴歸**：舊版測試的分數是否下降？(不佳，需要調查)。
 - **能力**：在新測驗中，分數是否有所提升？(很好，這是進展)。

2. 失敗分析：請不要只看分數，

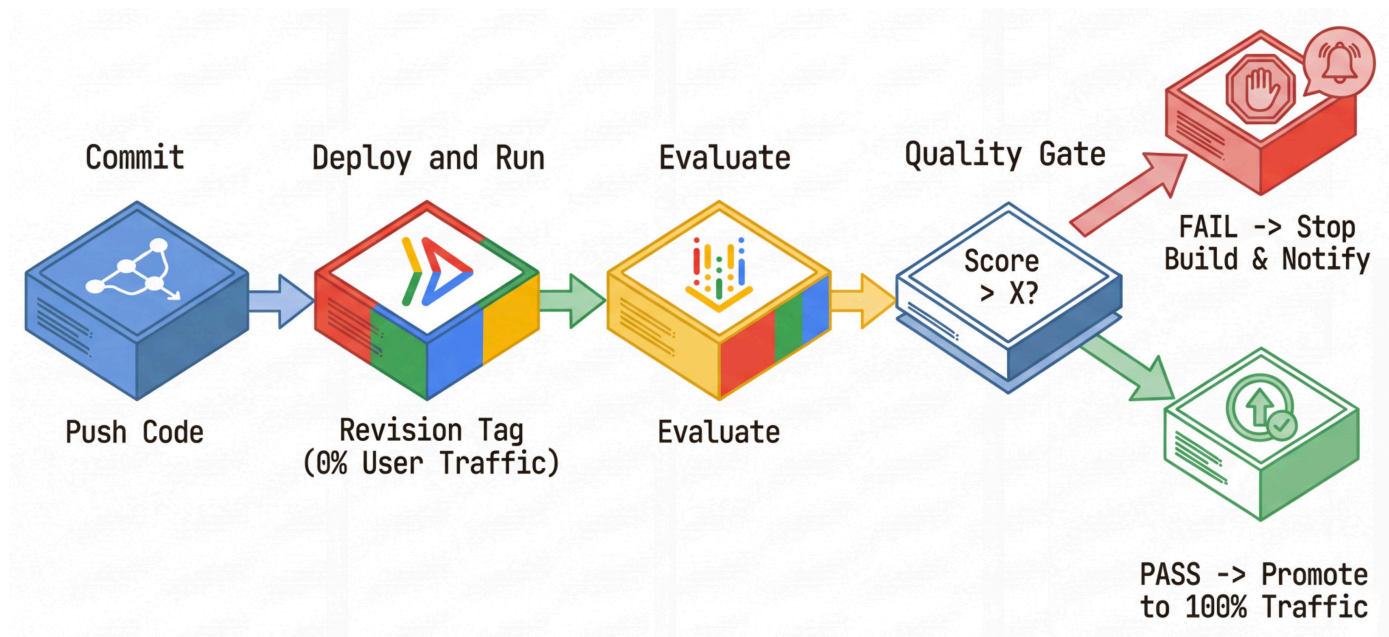
- 查看 *追蹤記錄*。是否呼叫了錯誤的工具？是否無法剖析輸出內容？你可以在這裡找到昆蟲。
- 查看法官 LLM 提供的說明和評定結果。通常可以幫助您瞭解測試失敗的原因。

Pass@1 與 Pass@k：執行特定測試一次時，我們會取得 Pass@1 分數。如果代理程式失敗，可能是因為非決定性。在複雜的設定中，您可能會執行每項測試 `k` 次 (例如 5 次)，並計算 `pass@k` (是否至少成功一次？) 或 `pass^k` (是否每次都成功？)。許多指標在內部運作時，就是採用這種做法。舉例來說，`types.RubricMetric.FINAL_RESPONSE_MATCH` (最終回應相符) 會對評估 LLM 發出 5 次呼叫，以判斷最終回應相符分數。

9. 持續整合與部署 (CI/CD)

在實際工作環境系統中，應將代理程式評估做為 CI/CD 管道的一部分執行。[Cloud Build](#) 是不錯的選擇。

每當有提交內容推送至代理程式的程式碼存放區，評估作業就會與其餘測試一併執行。如果通過，即可「升級」部署作業，開始處理使用者要求。如果測試失敗，一切都會維持原狀，但開發人員可以查看問題所在。



Cloud Build 設定

現在，讓我們建立 Cloud Run 部署設定指令碼，執行下列步驟：

1. 將服務部署至私人修訂版本。
2. 執行代理評估。
3. 如果評估通過，系統會「升級」修訂版本部署作業，將 100% 的流量導向該版本。

建立 `cloudbuild.yaml`：

```

steps:
- name: gcr.io/google.com/cloudsdktool/google-cloud-cli:latest
  entrypoint: /bin/bash
  env:
    - 'BUILD_ID=$BUILD_ID'
  args:
    - "-c"
    - |
      if [[ "$_COMMIT_SHORT_HASH" != "" ]]; then
        export COMMIT_SHORT_HASH=$_COMMIT_SHORT_HASH
      else
        export COMMIT_SHORT_HASH=$SHORT_SHA
      fi
      export COMMIT_REVISION_TAG="c-$$${COMMIT_SHORT_HASH}"
      echo "Deploying with revision tag: $$COMMIT_REVISION_TAG"
      set -e
      # Install uv and sync dependencies.
      curl -LsSf https://astral.sh/uv/install.sh | sh
      source $$HOME/.local/bin/env
      uv sync

      # Deploy services with the revision tag.
      source ./deploy.sh --revision-tag $$COMMIT_REVISION_TAG --no-redeploy

      # Run evaluation.
      uv run -m evaluator.evaluate_agent
      # If evaluation fails, the deployment will stop here.

      # If evaluation passes, it will continue with promoting the revisions to serve 100%
of traffic.
      echo "Promoting revisions $$COMMIT_REVISION_TAG to serve 100% of traffic."
      gcloud run services update-traffic researcher --to-tags $$COMMIT_REVISION_TAG=100 --
region $$GOOGLE_CLOUD_REGION --project $$GOOGLE_CLOUD_PROJECT
      gcloud run services update-traffic judge --to-tags $$COMMIT_REVISION_TAG=100 --region
$$GOOGLE_CLOUD_REGION --project $$GOOGLE_CLOUD_PROJECT
      gcloud run services update-traffic content-builder --to-tags
$$COMMIT_REVISION_TAG=100 --region $$GOOGLE_CLOUD_REGION --project $$GOOGLE_CLOUD_PROJECT
      gcloud run services update-traffic orchestrator --to-tags $$COMMIT_REVISION_TAG=100 -
-region $$GOOGLE_CLOUD_REGION --project $$GOOGLE_CLOUD_PROJECT
      gcloud run services update-traffic course-creator --to-tags $$COMMIT_REVISION_TAG=100
--region $$GOOGLE_CLOUD_REGION --project $$GOOGLE_CLOUD_PROJECT

options:
  substitutionOption: 'ALLOW_LOOSE'
  defaultLogsBucketBehavior: REGIONAL_USER_OWNED_BUCKET

```

執行管道

最後，我們可以執行評估管道。

在執行評估管道 (向 Cloud Run 服務發出要求) 之前，我們需要具備多項權限的獨立服務帳戶。現在來編寫指令碼，執行這項作業並啟動管道。

1. 建立指令碼 `run_cloud_build.sh` :

```

#!/bin/bash

set -e
source .env

BUILD_SA_NAME="agent-eval-build-sa"
BUILD_SA_EMAIL="${BUILD_SA_NAME}@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com"
COMMIT_SHORT_HASH=$(git rev-parse --short HEAD)

# Creating service account for Cloud Build. Granting necessary roles to it.
if ! gcloud iam service-accounts describe "${BUILD_SA_EMAIL}" --project
"${GOOGLE_CLOUD_PROJECT}" &> /dev/null; then
    echo "Creating service account ${BUILD_SA_NAME} for Cloud Build."
    gcloud iam service-accounts create ${BUILD_SA_NAME} --project
"${GOOGLE_CLOUD_PROJECT}" --display-name "Agent Build Service Account"

    echo "Granting roles to service account ${BUILD_SA_NAME}..."
    sleep 10
    ROLES=(
        "roles/cloudbuild.builds.builder"
        "roles/run.admin"
        "roles/run.invoker"
        "roles/iam.serviceAccountOpenIdTokenCreator"
        "roles/iam.serviceAccountUser"
        "roles/serviceusage.serviceUsageAdmin"
        "roles/serviceusage.serviceUsageConsumer"
        "roles/aiplatform.user"
        "roles/logging.logWriter"
        "roles/storage.admin"
        "roles/artifactregistry.writer"
    )

    # Loop through and grant each role
    for ROLE in "${ROLES[@]}"; do
        gcloud projects add-iam-policy-binding "$GOOGLE_CLOUD_PROJECT" \
            --member="serviceAccount:$BUILD_SA_EMAIL" \
            --role="$ROLE" \
            --condition=None \
            --quiet
    done

    echo "Waiting for 60 seconds for the permission changes to propagate..."
    sleep 60
fi

gcloud builds submit --config cloudbuild.yaml \
    --service-
account="projects/${GOOGLE_CLOUD_PROJECT}/serviceAccounts/${BUILD_SA_EMAIL}" \
    --machine-type=e2-highcpu-32 \
    --timeout=120m \
    --substitutions
_COMMIT_SHORT_HASH=$_COMMIT_SHORT_HASH,_GOOGLE_CLOUD_PROJECT=$_GOOGLE_CLOUD_PROJECT,_GOOGLE

```

```
_CLOUD_LOCATION=$GOOGLE_CLOUD_LOCATION, _GOOGLE_CLOUD_REGION=$GOOGLE_CLOUD_REGION
```

這個指令碼：

- 建立專用服務帳戶 `agent-eval-build-sa` 。
- 授予必要角色 (`roles/run.admin` 、 `roles/aiplatform.user` 等) 。*. 將建構作業提交至 Cloud Build 。

2. 執行管道：

```
chmod +x run_cloud_build.sh  
./run_cloud_build.sh
```

您可以在終端機中查看建構進度，或按一下連結前往 Cloud Console 。

注意：在實際的正式環境中，您會設定 **Cloud Build 觸發條件**，在每次 `git push` 時自動執行這項作業。工作流程相同：觸發條件會執行 `cloudbuild.yaml`，確保評估每個提交內容。

10. 摘要

您已成功建構評估管道！

- **部署**：您使用修訂版本標記和 Git 提交雜湊，將代理程式安全地部署至實際環境進行測試，不會影響實際部署作業。
- **評估**：您定義了評估指標，並使用 Vertex AI Gen AI Evaluation Service 自動執行評估程序。
- **分析**：您使用 Colab 筆記本將評估結果視覺化，並改善代理程式。
- **推出**：您使用 Cloud Build 自動執行評估管道，並將最佳修訂版本升級，以處理 100% 的流量。

這個週期「編輯程式碼」->「部署標記」->「執行評估和測試」->「分析」->「推出」->「重複」是「生產級代理程式工程」的核心。
